

Joint Optimization With DNN Partitioning and Resource Allocation in Mobile Edge Computing

Chongwu Dong, Sheng Hu, Xi Chen, Wushao Wen*

School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China
{dongchw3, wenwsh}@mail.sysu.edu.cn, {hush26, guox33, chenx636}@mail2.sysu.edu.cn

Abstract—With the rapid development of computing power and artificial intelligence, IoT devices equipped with ubiquitous sensors are gradually installed with intelligence. People can enjoy many conveniences with intelligent devices, such as face recognition, video understanding, and motion estimation. Currently, deep neural networks are the mainstream technology in intelligent mobile applications. Inspired by DNN model partition schemes, the paradigm of edge computing could be utilized collaboratively to improve the effectiveness of intelligent task execution in IoT devices. However, due to the dynamics of the wireless network environment and the increasing number of IoT devices, a DNN partition policy without adequate consideration would pose a significant challenge to the efficiency of task inference. Moreover, the shortage and high rental cost of edge computing resources make the optimization of DNN-based task execution more difficult. To cope with those situations, we propose a joint method by a self-adaptive DNN partition with cost-effective resource allocation to facilitate collaborative computation between IoT devices and edge servers. Our proposed online algorithm can be proved to ensure the overall rental cost within an upper bound above the optimal solution while guaranteeing the latency for DNN-based task inference. To evaluate the performance of our strategy, we conduct extensive trace-driven illustrative studies and show that the proposed method can achieve sub-optimal results and outperforms other alternative methods.

Index Terms—Computation offloading, Lyapunov Optimization, edge intelligence, mobile edge computing, deep learning

I. INTRODUCTION

With the prevalence of 5G networks and the fast growth of artificial intelligence (AI), intelligent applications in user's portable equipment and other embedded terminals are becoming more and more popular. These intelligent applications are mainly implemented by deep learning, in which Deep Neural Networks (DNN) [1] plays a significant role in many fields, such as autonomous driving [2] and industrial manufacturing [3]. The architecture of DNN-based smart applications is composed of sequential and cascading layers, in which a massive amount of matrix operations are the main calculation type. In the case of the lightweight model, the total computation consumption in the DNN-based application could be undertaken by IoT devices. With the increasing complexity of scenarios and the higher demand for model accuracy, DNN-based inference is becoming heavier and computation-intensive. Even though the computation of DNN-based inference is far less than that of the DNN-based training process,

an IoT device's capacity with limited resources could not hold such tasks. For example, one wearable device (Google Glass, etc.) could not undertake a typical DNN-based inference due to its tiny physical space. Despite the development of hardware specialization [4] and IoT device performance, it is still a big challenge to handle the DNN-based inference in IoT devices due to the insufficiency of computing resources and battery life limitation in terminals.

To mitigate the above problems in IoT devices, task offloading to the cloud [5] [6] has been proposed to make up for the shortage of computing resources in devices. During task offloading, the source data for AI inference should be uploaded into the remote cloud over the Internet. Data transmission latency incurred by this process is decided by the wireless network and the amount of data transmission. Thus, it can be concluded that the service node close to devices, rather than a remote cloud, could significantly reduce the network backhaul latency to improve the interaction and experience of intelligent service.

With the advantage of location proximity to IoT devices, edge computing [7] is utilized to facilitate the deployment of AI. This emerging solution, called edge intelligence (EI) [8] [9], is pervasive and attracts much attention in academia and industry. The key idea of EI is to allow smart devices to offload their local intelligent tasks to edge clouds in proximity to the network edge. By utilizing the approach of computation partitioning, [10], the execution of offloading tasks could be accelerated and get a real-time response with the help of edge clouds. Besides, DNN-based application can partition its model and be partially handled on both local devices and edge clouds in a collaborative pattern, instead of uploading the entire task [11]. According to existing statistical work for DNN [12] [13] [14], some intermediate features could be quite smaller than that of the raw source data. In this case, data transmission can be reduced by partitioning the DNN model and offloading the intermediate feature to edge clouds. So, with the assistance of DNN partition, the advantage of EI can be more fully utilized to improve the performance of DNN-based task inference.

However, there are still lots of challenges for the optimization of DNN-based task inference in the EI environment. Although an edge cloud has more sufficient resources than that in IoT devices, the quality of AI service could be seriously degraded when the edge cloud could not undertake excessive

computation beyond its capacity [15]. Practically, the available resources for task offloading, including computation in an edge cloud and bandwidth in the wireless network, would change a lot over different time slots [16]. Indeed, the arrival of user requests could not be predicted. Without full consideration, the splitting decision on the DNN partition may lead to a significantly worse deceleration for AI inference. Besides, the performance of DNN-based task execution can also be affected by many other factors [17] [18], one of which is different types of computing auxiliary equipment, such as CPU, GPU, and TPU. In fact, the heterogeneity of hardware specialization has inconsistent effects on DNN inference. For instance, GPU has more advantages over CPU in large-scale DNN inference [19], due to that parallel computing is more suitable for GPU architecture. Some proposed studies, such as [20] [21], only considered one single type of resource to participate in the process of task acceleration, which would not be practical. Generally, It is more expensive to construct one edge node than renting computing resources. In this case, service providers are inclined to rent resources from edge service vendors. However, higher quality and quantity of resource consumption may incur more financial cost pressure. Based on this issue, some studies [22] [23], which distributed the highest and most quantity resources for each offloading task to reduce task inference latency, would cause substantial economic costs. In fact, users, under the upper bound of task inference delay, would achieve a similar service experience even spending higher rental costs with more resources. So, it is significant and urgent to design a self-adaptive and cost-effective DNN model partition strategy to distribute heterogeneous computations for intelligent mobile applications in the dynamic and unpredictable service environment.

In this paper, we tackle these challenges by a joint strategy based on Lyapunov theory [24], in which our method decides the model partition point while collaborating with resources allocation in an edge cloud. On the side of devices, our strategy considers ensuring the upper bound on the task inference delay. On account of edge cloud, the economics of resource allocation are taken into consideration in our strategy for computation acceleration. In summary, our main contributions in this work are as follows:

- We investigate the acceleration of DNN-based task inference in a mobile edge cloud (MEC) equipped with heterogeneous resources, as illustrated in Figure 1. The heterogeneous computation resources with different capacities and rental costs are introduced to describe the model of MEC. One constrained stochastic optimization is formulated to achieve computation efficiency for the local device and cost-effectiveness for the edge cloud. To the best of our knowledge, this work is the first one to consider both computation efficiency and cost-effectiveness to accelerate DNN-based task acceleration in the MEC.
- We develop an online joint algorithm based on the Lyapunov optimization framework, which does not depend

on future information, such as the network condition and the arrival of user requests. Our online algorithm can reduce the rental cost by proper resource allocation while guaranteeing computation efficiency for IoT devices. We further study the properties of our algorithm and show that it can achieve a suboptimal result within a provable upper bound from the optimal solution.

- We conduct trace-driven illustrative studies to evaluate the performance of our proposed algorithm. The results show that our proposed algorithm can reduce the rental costs by up to 30% for AI inference computation while ensuring high-quality performance for DNN-based task inference.

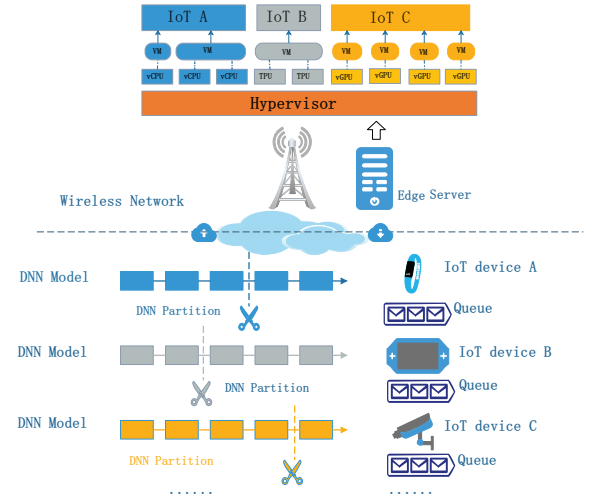


Fig. 1. The Architecture of MEC Cloud Platform: DNN-based Task Acceleration by Model Partition and Resource Allocation Jointly.

II. RELATED WORK

According to the report¹ from Garner, DNN-based innovative application plays a significant role in edge intelligent (EI) service, which has attracted lots of attention from industrial and academic. To improve the quality of EI service, DNN Model partitioning has been proposed [25] [26] to accelerate EI inference and alleviate computing pressure on IoT equipment. Among the recent studies for DNN model partition, the DNN model is summarized into two different types. One is formulated as a directed graph [12], while the other one is simplified as a linked list [27]. Based on the setting of two network structures, DNN inference can be partitioned between IoT devices, and remote edge servers, such as [28] [29]. Kang *et al.* [22] firstly proposed one model partitioning framework between local devices and an edge cloud, considering the regression model to predict the computation cycles of each layer in DNN. Ran *et al.* [30] considered utilizing a numerical method, which depends on the amount of parameter size, to predict the consumption of computation for each DNN layer.

¹<https://www.gartner.com/document/3988118>

Similarly, with previous work, Ko *et al.* [14] also utilized a static method to measure the inference time in a fixed deployment environment. Based on the lots of future knowledge about the environment, this work designed different strategies to decide an appropriate partitioning point. However, the insufficient resources and performance fluctuation in an edge cloud could significantly influence the accuracy of prediction results, which could not be predicted. So, the computation acceleration strategy for DNN-based task inference, without considering information unavailable, is hard to cope with the dynamics of the environment.

Besides, there is some work about general partial offloading optimization [31], which is usually formulated as an ongoing problem. This kind of problem can obtain the optimal result quickly by convex optimization. In fact, DNN partitioning belongs to the discrete problem. So, it is necessary to explore new approaches to solve this new type of problem, which is now very popular in academia and industry.

To formulate the problem of DNN model partitioning in the MEC platform, recent studies treated task inference latency as the main optimization goal on the AI service quality. For example, Li *et al.* [32] and Eshratifar *et al.* [33] formulated the problem of model partitioning between one device and an edge cloud, in which the data transmission latency is the key factor to affect the performance of model partitioning. In their optimization formulation, the device could obtain a sufficient amount of computing resources. However, the edge cloud could not support AI services properly when too many users are arriving simultaneously. Tang *et al.* [34] and Mohammed *et al.* [27] concerned the edge cloud, in which equipment is thought homogeneously without distinction. They utilized the best-effort strategy of resource allocation for model partitioning. In fact, the acceleration performance and rental cost in heterogeneous computation resources [35] [36] are diverse. The best-effort strategy of resource allocation without considering heterogeneity may cause great pressure on the rental cost. That is to say, it is essential to consider heterogeneous resource allocation on demand, aiming to design one cost-effective partitioning point selection strategy.

Considering joint optimization on both cost and resource, there exist many studies in other fields, such as Virtual Network Embedding [37], that concentrate on this type of strategy. Yao *et al.* [38] and Yan *et al.* [39] proposed two similar methods based on deep reinforcement learning to improve the network throughput under dynamic environment. It is noted that all the aforementioned approaches about Virtual Network Embedding mainly focus on network resource utilization. However, in the scenario of Mobile Edge Computing, the computing resources is another essential part of the optimization, which should not be neglected.

To the best of our knowledge, this work differs from previous works in the following: Although there are some existing studies about DNN inference acceleration by model partitioning in the MEC system, these studies mainly considered the effect of task inference latency on the side of IoT equipment. Their proposed strategies are based on lots

TABLE I
LIST OF NOTATIONS

Symbol	Description
$\mathcal{K}$	The Set of IoT devices
$\mathcal{L}$	The Computation capacity set of IoT devices
S	The number of a hardware type
P	The Computation capacity set of hardware in the edge
C	The Cost of each hardware for one hardware Type
$\mathcal{E}$	The partitioning point set in the DNN model
s	The Point Selection for different devices
d	The number of different types of allocated resources
T	The latency for different process

of future knowledge. Unlike these work, we consider cost-effectiveness into the decision of model partitioning point by heterogeneous resource allocation on demand. In addition, based on the Lyapunov theory, our proposed method can dynamically adjust with minimum information exchange, which fits significantly in a dynamic edge cloud. Our algorithm is proved to achieve the sub-optimal result within an upper bound to the optimal solution by theoretical analysis.

III. SYSTEM MODELS AND PROBLEM FORMULATION

In this section, we explain DNN-based inference acceleration for multiple IoT devices in a MEC platform. First, we introduce the architecture of a MEC cloud platform with one edge cloud and multiple IoT devices, just as illustrated in Fig. 1. The edge cloud is deployed in proximity to a base station, which has various kinds of hardware devices for computation acceleration, such as GPU and CPU. In each IoT device, DNN-based tasks can be partitioned into two parts for offloading and local computation. Each partial offloading task can be allocated with resources on demand for acceleration. Each IoT device and the edge cloud both deploy the entire DNN model for processing. Then, we introduce the feature of the DNN model. By the method of DNN model partitioning, those IoT devices can offload their partial intelligent inference to the edge cloud through the cellular network. Next, heterogeneous resource allocation for each task in the edge cloud is introduced, which would have an essential effect on inference acceleration and rental cost reduction. After that, a model for the integration between partitioning point decisions and heterogeneous resource allocation is introduced. Necessary notations are list in the table I to improve the understanding of our problem formulation.

A. System Model

In our formulation, IoT devices can offload partial computation by DNN model partitioning, in which some intermediate feature data must go through the wireless network. Moreover, we denote the bandwidth of the access network as B . The bandwidth is set to be shared equally among all devices, in which the set of IoT devices with the total number v is denoted as $\mathcal{K} = \{k_1, k_2, k_3, \dots, k_v\}$. The computation capacity set of IoT devices can be denoted as $\mathcal{L} = \{l_1, l_2, l_3, \dots, l_v\}$. Each IoT device would generate a series of task inferences along with different time slots.

In an edge cloud, heterogeneous hardware can provide different degrees of computation acceleration. To improve hardware utilization efficiency, cloud service providers (CSP) generally use virtualization techniques, such as virtual machines and execution containers, to allocate computation resources in a fine-grained way. Due to the inconsistency of software support among heterogeneous hardware, the computation capacity among different hardware devices types should be formulated independently and respectively. Traditionally, virtualization technology, such as KVM, can divide one CPU into several parts by the number of computing cores. Besides, there is an extra burden put into the edge cloud, which is set similarly according to the previous study [40]. In this case, the idle computing resources that can be allocated in the edge cloud should be set dynamically. So, the number of different types of devices in each time slot is set unfixed. And, we define $\mathcal{S}_{cpu}^t = \{cpu_1^t, cpu_2^t, cpu_3^t, \dots, cpu_n^t\}$ as the set of CPU type in an edge cloud. Each element is denoted the number of CPU cores for different CPU types in the time slot t . $\mathcal{P}_{cpu} = \{P_{cpu_1}, P_{cpu_2}, P_{cpu_3}, \dots, P_{cpu_n}\}$ is denoted as the set of computation capacity for each CPU core. However, other computing accelerators, such as GPU and TPU, have limited software virtualization support. Most CSPs provide computation acceleration service by Pass-Through, or Grid vGPU [41], in which one GPU could only be divided into a fixed number of virtual sub-devices. In the GPU sharing mode, CSPs should pay additional software licensing fees to provide a fixed number of hardware divisions for resource allocation. Considering the popular virtual technology, we define $\mathcal{S}_{gpu}^t = \{gpu_1^t, gpu_2^t, gpu_3^t, \dots, gpu_m^t\}$ as the set of GPU type, in which each element represents the number of virtual GPU (vGPU) for different types in the time slot t . Similarly, $\mathcal{P}_{gpu} = \{P_{gpu_1}, P_{gpu_2}, P_{gpu_3}, \dots, P_{gpu_m}\}$ is denoted as the set of computation capacity for each vGPU. In the meantime, we set tpu^t and $\mathcal{P}_{tpu}$ as the total number of TPU in the time slot t and the computation capacity of each TPU, respectively. Practically, the number of vGPU in each type is less than the CPU in an edge cloud. Given the similarity setting between GPU, TPU, and other accelerators, we can simplify our problem formulation by ignoring other accelerators in our formulation.

There is quite a lot of differences in the resource rental cost for different types of hardware. According to the pricing model from CSPs, such as Aliyun² and ECloud³, our work utilizes the model of pay-as-you-go to denote the resource rental cost for AI inference acceleration. We assume that our EI service runs in a time-slotted style, in which time is equally divided into a series of time slots. The intermediate data incurred by model partitioning should pass through the access network, which is not free. However, the amount of offloading data is tiny enough that it would not cause much transmission cost. So, the data transmission cost is neglected in our formulation. So, in each time slot, the total cost for AI inference acceleration

mainly consists of GPU, TPU, or CPU rental cost. And, we define the unit cost for renting each type of single-core CPU as $\mathcal{C}_{cpu} = \{C_{cpu_1}, C_{cpu_2}, C_{cpu_3}, \dots, C_{cpu_n}\}$, the unit cost for renting each type of vGPU device as $\mathcal{C}_{gpu} = \{C_{gpu_1}, C_{gpu_2}, C_{gpu_3}, \dots, C_{gpu_m}\}$, and the similar setting to the unit cost for TPU as $\mathcal{C}_{tpu}$.

B. DNN Partitioning Model

The architecture of the DNN network can be treated as a composition of sequential logical layers, similar to previous work [34]. As shown in Fig. 2, the internal structure of every layer is composed of multiple neurons. These essential elements inside each layer are equipped with many parameters, which participate in various scientific operations, such as convolution. Although a directed graph can be introduced to formulate the fine-grained relationship between different neurons inside logical layers, the problem complexity would increase exponentially. In this case, our work abstracts DNN architectures as a linked list composed of different logical layers and edges.

The edge set between logical layers in the DNN model is denoted as $\mathcal{E} = \{e_1, e_2, e_3, \dots, e_f\}$, representing the location of model partitioning point. When the point is set to be e , the computation volume of the DNN model in the IoT device and edge cloud can be denoted as X_e^{local} and X_e^{edge} , respectively. The output data that need to be uploaded is defined as D_e . Due to the small scale of DNN inference results, we can ignore the effect of data transmission from an edge cloud to IoT devices.

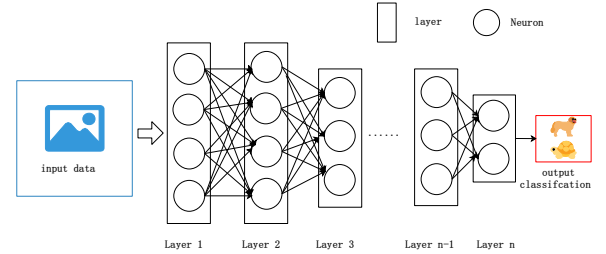


Fig. 2. An Example For Image Classification: a typical DNN Model with multiple sequential layers

C. Problem Formulation

In IoT devices, the model partitioning point for DNN-based tasks is the location that can decide the amount of computation in an edge cloud. In the edge cloud, heterogeneous computation resource allocation can decide the efficiency of each offloaded task execution. The above two optimization decisions can be jointly utilized to save the total operational cost and ensure task inference performance, following the constraints of the service environment and optimization problem. In each time slot, we assume that the above decisions hold within a time slot, which can be formulated as follows:

(1) $s_k(e, t)$ is denoted as one indicator to describe the selection of model partitioning point for the IoT device k in the time slot t . And, $s_k(e, t)$ should meet the constraint of $s_k(e, t) \in \{0, 1\}$. In each point e , $\{D_e, X_e^{local}, X_e^{edge}\}$ could

²<https://developer.aliyun.com/article/661790>

³<https://www.ctyun.cn/price/index>

be utilized to show the process of computation offloading to an edge cloud when model partitioning point is set to be e .

(2) $\theta_k = \{d_k^c(n, t), d_k^g(m, t), d_k^u(t)\}$ is set to be a notation to represent the resource allocation strategy for IoT device k . Θ is denoted as the set representing all resource allocation strategy. $d_k^c(n, t)$ is denoted as an integer variable to represent the number of CPU core of type n allocated to accelerate computation to IoT device k . $d_k^g(m, t)$ and $d_k^u(t)$ are denoted similarly with $d_k^c(n, t)$, to show the number of type m vGPU and TPU for IoT device k , respectively. $\mathbb{I}_k = \{I_k^c(n, t), I_k^g(m, t), I_k^u(t)\}$ is denoted as an indicator to show whether corresponding accelerator is allocated to IoT device k .

Task Inference Latency. From the perspective of IoT devices, task execution delay, has a major effect on the quality of DNN inference. The offloading process for each individual task is set similarly to the previous work [27], including three independent stages. In fact, each task can be processed in parallel, just as [12] [42]. These studies treated the task offloading in the pipeline, owing to that tasks are queued at the edge cloud. In our work, we neglected the task queuing process by considering that each task can be held by one independent container or VM. We can obtain that the inference between different tasks is small. So, we treat the task processing can be nearly treated as a sequential mode. That is to say, during the process of DNN partitioning, the latency mainly consists of three parts, including local execution delay, data uploading delay, and edge-assistance computation delay. Other latency, including the round-trip and downlink delay between devices and edge cloud, is a small constant related to the distance. In this case, we can ignore other non-essential factors in our optimization. Thus, we can denote task execution delay $T_{k,e}^d$ for IoT device k selecting the partitioning point e and adopting the resource allocation strategy θ as below:

$$T_{k,e}^\theta = T_{k,e}^{edge} + T_{k,e}^{up} + T_{k,e}^{local} \quad (1)$$

in which $T_{k,e}^{local}$ is denoted as local execution delay, $T_{k,e}^{up}$ as data uploading delay and $T_{k,e}^{edge}$ as edge-assistance execution delay. And, we can calculate the value of each delay by the following expression as below:

$$T_{k,e}^{edge} = \frac{\mathcal{X}_e^{edge}}{P_{k,t}^{edge}} \quad (2)$$

$$T_{k,e}^{up} = \frac{D_e}{B_{k,t}} \quad (3)$$

$$T_{k,e}^{local} = \frac{\mathcal{X}_e^{local}}{l_k} \quad (4)$$

In the above expression, $B_{k,t}$ is set as the bandwidth allocated to device k in the time slot t , which would dynamically change along with the deployment environment. And, $P_{k,t}^{edge}$ is denoted as the edge computation resource allocated to device k in the time slot t , in which we can calculate it by utilizing the expression (5) as below:

$$P_{k,t}^{edge} = \sum_{n \in \mathcal{N}} d_k^c(n, t) P_{cpu_n} + \sum_{m \in \mathcal{M}} d_k^g(m, t) P_{gpu_m} + d_k^u(t) P_{tpu} \quad (5)$$

The average delay t_{avg} for all IoT devices in the time slot t could be represented as below:

$$T_{avg}^t = \sum_{k \in \mathcal{K}} \frac{T_{k,e}^\theta}{v} \quad (6)$$

Rental Cost. In each time slot, the cost for AI inference acceleration is incurred by rental computation resource, which can be described as below:

$$Cost_{k,t}^\theta = \sum_{n \in \mathcal{N}} d_k^c(n, t) C_{cpu_n} + \sum_{m \in \mathcal{M}} d_k^g(m, t) C_{gpu_m} + d_k^u(t) C_{tpu} \quad (7)$$

Then, we can obtain the total rental cost for all IoT devices at the time slot t , which can be described as below:

$$Cost_{total}^t = \sum_{k \in \mathcal{K}} Cost_{k,t}^\theta \quad (8)$$

D. Cost-Minimization Problem.

For IoT devices that resort to an edge cloud for computation acceleration, the critical challenge is to minimize the total rental cost while ensuring the QoE for AI inference. To achieve this goal, we utilize the following stochastic optimization problem to formulate this challenge:

$$\mathbf{P1.} \min \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}} Cost_{k,t}^\theta \quad (9)$$

The feasible solution to solve the minimization of Problem **P1** should consider the constraint of resource limitation in an edge cloud. Besides, the average task inference execution delay T_{avg}^t should also be controlled under a reasonable upper bound so that the performance of the AI inference task could be guaranteed. In this case, the constraints in our optimization formulation can be described as below:

$$0 \leq \sum_{k \in \mathcal{K}} \sum_{n \in \mathcal{N}} d_k^c(n, t) \leq cpu_n \quad (10)$$

$$0 \leq \sum_{k \in \mathcal{K}} \sum_{m \in \mathcal{M}} d_k^g(m, t) \leq gpu_m \quad (11)$$

$$0 \leq \sum_{k \in \mathcal{K}} d_k^u(t) \leq tpu \quad (12)$$

$$I_k^u(t) + I_k^g(m, t) + I_k^c(n, t) \leq 1, \forall m \in \mathcal{M}, \forall n \in \mathcal{N} \quad (13)$$

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^T T_{k,e}^\theta \leq \epsilon, \forall k \in \mathcal{K} \quad (14)$$

$$T_{k,e}^\theta \leq T_k^{local}, \forall k \in \mathcal{K} \quad (15)$$

The constraint (10)(11)(12) indicate the quantitative boundaries among different heterogeneous resources, in which the total allocated resources could not exceed the available number of the edge cloud. The constraint (13) ensures that for each IoT device $k \in \mathcal{K}$, only one type of hardware could be utilized for task inference execution. The constraint (14) specifies that the average task inference delay for each IoT device should be bounded under a threshold ϵ . In the constraint (15), T_k^{local} is denoted as the local DNN inference time incurred by the local-only strategy for each IoT device k . The last constraint shows that collaborative task inference should not play a negative effect on task execution performance.

IV. ONLINE ALGORITHM

To solve the problem **P1** under the constraints of resource limitation and QoE assurance for DNN inference, we design an online strategy to dynamically decide the partitioning point and adaptively allocate heterogeneous resources based on the Lyapunov optimization framework [24]. Lyapunov optimization belongs to the control theory, which can ensure the stability of a dynamic system while optimizing the performance objective. The system usually utilizes a series of virtual queues to show the degree of dynamics. The stability of these virtual queues can be guaranteed by minimizing the drift of a quadratic Lyapunov function under the max-weight algorithm. So, the original performance object can be transformed by optimizing the drift-plus-penalty function. Compared with previous studies, such as numerical method [30] for computation consumption prediction and static bandwidth configuration without considering unexpected factors [43] [44], our online control algorithm does not depend on the future knowledge about the edge cloud and wireless bandwidth condition.

A. Problem Transformation

In the framework of Lyapunov optimization, Problem **P1** is first transformed into a problem of minimizing the Lyapunov Drift-Plus-Penalty function. We denote $Q = \{Q_1, Q_2, Q_3, \dots, Q_k\}$ as the virtual backlog set to represent the time average constraint (14). $Q_k(t)$ is set to be zero at the beginning of DNN collaborative inference (co-inference) between IoT device k and the edge cloud. And, the update of virtual queue backlog $Q_k(t)$ is presented as follow:

$$Q_k(t+1) = \max\{Q_k(t) + T_{k,e}^\theta - \epsilon, 0\} \quad (16)$$

By utilizing the Lyapunov optimization framework, the time average delay constraint (14) can be derived from the stability of virtual queue $Q_k(t)$. And, the following **lemma 1** gives this description with a proof:

Lemma 1. *If the virtual queue Q_k is stable, the average DNN co-inference latency for each IoT device k can satisfy the constraint (14), which can be described as below:*

$$\lim_{t \rightarrow \infty} \frac{Q_k(t)}{t} = 0 \implies \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} T_{k,e}^\theta \leq \epsilon \quad (17)$$

Proof: Please see Appendix A for the proof detail. ■

According to the Lyapunov framework [24], we denote the Lyapunov function as $\Phi(t) = \frac{1}{2} \sum_{k \in \mathcal{K}} Q_k(t)^2$. The Lyapunov drift function is denoted as $\Delta(Q(t)) = \Phi(t+1) - \Phi(t)$, representing the expected change in the Lyapunov function within one time slot. $Q(t)$ is denoted as the backlog matrix about all IoT devices. By the theory of the Lyapunov optimization framework, the drift-plus-penalty function can both minimize the total rental costs and satisfy time-average delay constraint.

$$\Delta(Q(t)) + \beta E(Cost_{total}^t | Q(t)), \quad (18)$$

where β is a tunable parameter to control the tradeoff between the stability of the virtual delay queue and the minimization of the total rental cost. If the stability of $Q(t)$ is more important than the minimization of the total rental cost, we can control it by decreasing the value of parameter β . Then, the original problem **P1** can be transformed into the problem **P2**:

$$\begin{aligned} \mathbf{P2.} \quad & \min \Delta(Q(t)) + \beta E(Cost_{total}^t | Q(t)) \\ \text{s.t.} \quad & (10) - (13) \end{aligned} \quad (19)$$

Based on the Lyapunov optimization framework, the problem **P2** can be solved by minimizing its upper bound rather than seeking an immediate solution. Therefore, the key is to find an upper bound on problem **P2**. The following **lemma 2** gives the upper bound of the drift-plus-penalty function with a proof.

Lemma 2. *Under any feasible set including partitioning point selection $s_k(e, t)$ and resource allocation strategy $\theta_k = \{d_k^c(n, t), d_k^g(m, t), d_k^u(t)\}$ for all IoT devices, the drift-plus-penalty function **P2** is upper bounded by an expression, which is described as follows:*

$$\begin{aligned} & \Delta(Q(t)) + \beta E(Cost_{total}^t | Q(t)) \\ & \leq \Gamma + \sum_{k \in \mathcal{K}} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon | Q(t)\} \\ & \quad + \beta E(Cost_{total}^t | Q(t)), \end{aligned} \quad (20)$$

where $\Gamma = \frac{1}{2} \sum_{k \in \mathcal{K}} (T_{k,e}^\theta - \epsilon)^2$ is a constant. T_k^{local} is set to be the task inference time incurred by the local-only strategy for IoT device k .

Proof: Please see Appendix B for the proof detail. ■

Instead of directly minimizing the drift-plus-penalty problem **P2**, we can minimize the right-hand-side (R.H.S) of (20), which is the upper bound of Problem **P2**. Hence, we can transform the problem **P2** into the problem **P3** as follows:

$$\begin{aligned} \mathbf{P3.} \quad & \min \sum_{k \in \mathcal{K}} Q_k(t) \cdot \{T_{k,e}^\theta - \epsilon\} + \beta Cost_{total}^t \\ \text{s.t.} \quad & (10) - (13) \end{aligned}$$

Because ϵ is a constant at each time slot, we can further simplify the minimization of **P3** as below:

$$\begin{aligned} \text{P4.} \quad & \min \sum_{k \in \mathcal{K}} Q_k(t) \cdot T_{k,e}^\theta + \beta Cost_{total}^t \quad (21) \\ \text{s.t.} \quad & (10) - (13) \end{aligned}$$

Based on the above problem transformation, we design an online algorithm in the next section, aiming at solving the minimization of problem **P4**. The algorithm can dynamically minimize the overall rental costs for computation acceleration while guaranteeing the quality of task collaborative inference with the assistance of an edge cloud.

B. Online Algorithm

By analyzing the problem **P4** and its constraints, our goal is to minimize the upper bound of problem **P1** along with different time slots. The Problem **P4** is a typical integer linear programming (ILP), which belongs to an NP-hard problem. This problem can only get a sub-optimal solution in polynomial time, such as the shortest path solution. Thus, the minimization in each slot cannot be obtained in time, which is determined by policy space set about the combination of the resource allocation $\theta_k(t)$ in the edge cloud and partitioning point selection $s_k(e, t)$ in the local IoT device.

Note that the information about $s_k(e, t)$ from IoT devices and $\theta_k(t)$ from the MEC server needs to be exchanged with each other several times, which are located in two different places. When the set of $s_k(e, t)$ is given in IoT devices, the minimization of **P4** is decided by the set of $\theta_k(t)$. Under the given partitioning point selection, the solution to minimize the above expression can mainly focus on the resource allocation on the prior knowledge of DNN partitioning so that the computational complexity can be reduced greatly.

Inspired by the above issue, we can simplify the optimization of **P4** into two sequential stages. During the minimization of problem **P4**, the partitioning point can be decided in the first stage. After that, we can transform the original problem **P4** into the optimization of heterogeneous resource allocation, which can be solved by the solution to equation (22), as below:

$$\begin{aligned} \min \sum_{k \in \mathcal{K}} Q_k(t) \cdot T_{k,e}^{edge} + \beta Cost_{total}^t \quad (22) \\ \text{s.t.} \quad (10) - (13) \end{aligned}$$

The solution to resource allocation can be expressed in terms of variable $s_k(e, t)$, defined as $\Theta^* = f(s_k(e, t))$. To sum up, Problem **P4** can be equivalently reformulated as below:

$$\begin{aligned} \text{P5.} \quad & \min \sum_{k \in \mathcal{K}} Q_k(t) \cdot T_{k,e}^{f(s_k(e, t))} + \beta \sum_{k \in \mathcal{K}} Cost_{k,t}^{f(s_k(e, t))} \quad (23) \\ \text{s.t.} \quad & (10) - (13) \end{aligned}$$

where the virtual queue backlog in (16), can be acknowledged from the previous time slot.

On the basis of the above analysis, our work designs an iterative model partition and resource allocation strategy based on a genetic algorithm (IMPAC) to get the sub-optimal result of Problem **P4**. At the beginning of each time slot t , our algorithm would iteratively the process of joint optimization for DNN partitioning point selection and resource allocation, based on historical information from the previous time slot. In each iteration, we first decide the selection of DNN partitioning points for all IoT users. Then, our algorithm can get one feasible solution to solve Problem **P5**. After several iterative steps, our algorithm would stop and get a sub-optimal solution to **P5** when the optimization of two successive evolution is almost the same, or the number of iterations i reaches one threshold value defined as σ .

Algorithm 1 Iterative Model Partition and Resource Allocation Strategy Based On Genetic Algorithm (IMPAC)

- 1: **Initialization:** For all IoT users, DNN model partition is randomly selected, and Let $t=0$;
 - 2: **while** The DNN-based Service is online: **do**
 - 3: **while** Two successive iterations are different or $i \leq \sigma$ **do**
 - 4: **for** $w \in W$ **do**
 - 5: Obtain θ_k by inputting the given DNN model partition $s_k(e, t)$ to the equation (22);
 - 6: calculate the result of problem **P5** by the known parameter $s_k(e, t)$;
 - 7: **end for**
 - 8: Select individual fitness-based solutions;
 - 9: Crossover and mutation operation;
 - 10: Update the next-generation W for all devices
 - 11: $i = i + 1$;
 - 12: **end while**
 - 13: Pick up the optimal joint strategy $s_k(e, t)$ and θ_k ;
 - 14: Update the virtual queue Q_k according to the equation (16);
 - 15: Update $t = t + 1$;
 - 16: **end while**
-

In the MEC platform, all IoT devices would exchange messages with the edge cloud. During the joint optimization process, the edge cloud would collect the information of all connected IoT devices and execute our iterative algorithm periodically. In our IMPAC algorithm, our work encodes the DNN partition strategy for all IoT users by binary coding. The expression of Problem **P5** is utilized as our fitness function. In the initialization of partitioning point selection for all IoT users, the edge-only, local-only, balance strategy and several random strategies are selected in our work, each of which is chromosome denoted as w . To ensure the child generation, individuals are selected to breed a new generation according to the fitness of all parent individuals. The crossover is set similarly with the previous work [45], which utilizes the standard single-point operation. The mutation is set as a random process in our work. After our algorithm updates the decision results of all IoT devices, these devices would obtain

their instructions and take actions according to the results. The detail of IMPAC is presented in algorithm 1 as below:

C. Algorithm and Complexity Analysis

To evaluate the performance of our algorithm IMPAC, we conduct a mathematical analysis to prove that IMPAC can achieve within an upper bound difference from the optimal operational costs.

Lemma 3. *The average expected total rental cost incurred by IMPAC can achieve the effect that has a gap with the minimal rental cost, which can be described as below:*

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} E(Cost_{total}^t) \leq Cost^* + \frac{\Gamma}{\beta},$$

where $Cost^*$ is the infimum of the average rental costs achieved by any joint strategy from the combination of DNN partition and resource allocation under the required constraints.

Proof: Please see Appendix C for the proof detail. ■

In fact, Although other partial offloading strategies that are not well designed can also achieve this bound, these strategies can not guarantee the stability of the virtual queue, which would not meet the delay constraint. That is to say, our algorithm can guarantee the stability of virtual delay queue $Q_k(t)$ for all IoT devices $k \in \mathcal{K}$.

Lemma 4. *The backlog of the virtual delay queue is stable, which is presented as follows:*

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{k \in \mathcal{K}} E(Q_k(t)) \leq \frac{\Gamma + \beta \cdot (\hat{Cost}_{total}^t - \check{Cost}_{total}^t)}{\delta},$$

where $\delta > 0$, $\hat{Cost}_{total}^t$ and $\check{Cost}_{total}^t$ are defined as the minimum and maximum rental costs in all strategies respectively, and Γ is equal to that in lemma 3.

Proof: Please see the Appendix D for the proof detail. ■

From the above two lemmas, we can acknowledge that the time-average total rental cost achieved by IMPAC is within a constant gap $\frac{\Gamma}{\beta}$ from the minimal rental cost, but also can be dynamically controlled by β . Moreover, the backlog of virtual queue $Q(t)$ can also be adjusted by the parameter β . The bigger value of β can reduce the total rental cost for resource allocation but decrease the stability of the virtual delay queue. Based on this property of our strategy, IMPAC can dynamically determine the value of β to meet different requirements from IoT devices. That is to say, when a burst of computing tasks arrive abruptly, the influx of many computational tasks can exhaust the computing resources in the edge cloud. When our method detects this situation of resource insufficiency, it will make timely adjustments in the next time slot, including reducing the amount of computation uploaded to the edge cloud. Our algorithm has sufficient robustness and self-adaptability to cope with this situation.

The time complexity of our algorithm IMPAC is determined by the loop of partitioning point selection. In the initial stage of our algorithm, a fixed number of a well-chosen point set is given for all IoT devices, which would participate in the evolutionary process until reaching to given iterative loop number or convergence. So, our algorithm can obtain the result in the time of $O(v\sigma)$, in which v represents the IoT devices' number, and σ is the given iterative number. Meanwhile, each loop of our evaluation is relatively independent and can be executed in parallel. This character can further reduce the complexity of IMPAC. To sum up, our algorithm could adapt to the dynamic of the practical environment and obtain the ideal result in polynomial time.

V. PERFORMANCE EVALUATION

In this work, we develop a discrete-event simulator based on YAFS [46] to simulate the environment of the MEC cloud platform and behaviors of IoT devices. To evaluate the performance of our iterative algorithm, our work conducts a set of trace-driven experiments to show the advantages of our strategy over other alternative strategies.

A. Experiment Setting

We assume that the MEC platform is deployed in a wireless network equipped with one edge node and a lot of IoT devices. In the simulated MEC platform, the number of intelligent IoT devices is set to be in the range of 10-50. To evaluate the adaptability of our algorithm, the elastic arrival, and departure of IoT devices are introduced in the different time slots, which follows the uniform distribution. In each IoT device, a DNN-based application is deployed, which generates a random number of DNN-based tasks for computation acceleration in each time slot. Moreover, we assume that the task generation process follows the Gaussian distribution. Besides, we choose AlexNet as a typical DNN model in the process of DNN inference acceleration. Similar to the literature [15], our work sets the configuration of computation and intermediate data for each layer of the DNN model from an online report⁴. The local computation capacity for IoT devices is set in the uniform distribution with parameters [0.5, 1] GHz, which is the reference of Raspberry pie III and IV⁵.

Following the real rental pricing model, we set the cost of a single-core CPU as \$0.09 per hour with computation capacity 0.04 TFLOPS, according to Tencent Cloud⁶. The rental cost of one GPU is set to be {\$0.99, \$1.20, \$1.86, \$1.98, \$2.73} per hour, corresponding to GPU type P4, M40, T4, P40, V100 equipped with computation capacity {5.5, 7, 8.1, 12, 15.7} TFLOPS. Each GPU can be sliced into 8 vGPU, corresponding to current technology, such as NVIDIA⁷. The rental cost of TPU is set as \$5.22/hour, with a computation configuration of 180 TFLOPS. We assume that one edge cloud is equipped with 32 vCPU. The number of GPU is set to be eight for each

⁴<https://github.com/LinBin403/dataset-for-our-research>

⁵<https://www.raspberrypi.org/products/>

⁶<https://buy.cloud.tencent.com/price/gpu>

⁷<https://www.nvidia.com/en-us/data-center/virtual-gpu-technology/>

type in this edge cloud. The number of TPU is also set to be eight in our simulation.

Following the similar network setting such as [47] and [5], the achievable data transmission rate $R(t)$ can be obtained by the formulation as below:

$$R(t) = B \log_2 \left(1 + \frac{h(t)P}{\theta^2} \right) \quad (24)$$

Generally, the bandwidth that each IoT device can obtain is related to the channel quality between the base station, the number of access devices, the IoT weight of resource allocation, etc. For simplification, we can set that the total frequency bandwidth 200MHz is shared among each IoT device transmission B , by the fair proportional method. The transmit power $P(t)$ for each IoT device is set to follow uniform distribution [0.05, 0.18] W, which would change along with different time slots. The parameter θ^2 is set to be 10^{-6} W. The channel gains $h(t)$ is set to follow the exponential distribution, where the mean is set to be 1.

B. Methodology

In the simulation, we compare three alternative strategies with our proposed IMPAC algorithm to verify the superior of our proposed strategy in terms of efficiency and stability. One extreme method, named Local-Only, is used to evaluate the performance boundary of our strategy. When the Local-Only method is used, IoT devices would only perform their DNN inference locally. Another two extreme methods, belonging to edge-only strategies, consider offloading the whole DNN inference to the edge cloud but allocate the computation resource of the edge cloud differently. The first kind of edge-only strategy, named EdgeBest, concentrates on inference acceleration performance without considering the cost. The second one, named EdgeOnDemand, would schedule resources on demand under the delay constraint. In the next section, we show the comparison of four strategies in our simulator to evaluate their performance. Task inference delay and rental cost are two primary metrics considered in our experiments to evaluate the stability and effectiveness of our strategy.

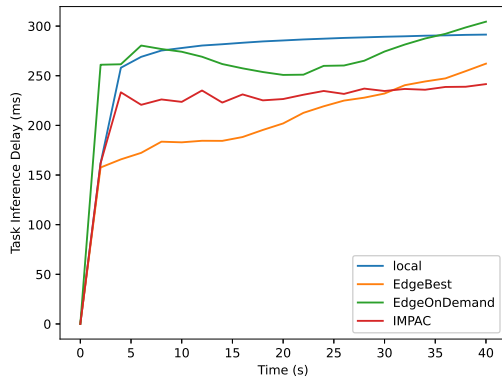


Fig. 3. Average Task Inference Delay Along With Different Time Slots

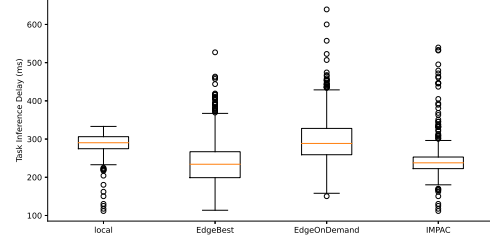


Fig. 4. Distribution of Task Inference Delay for Different IoT Devices

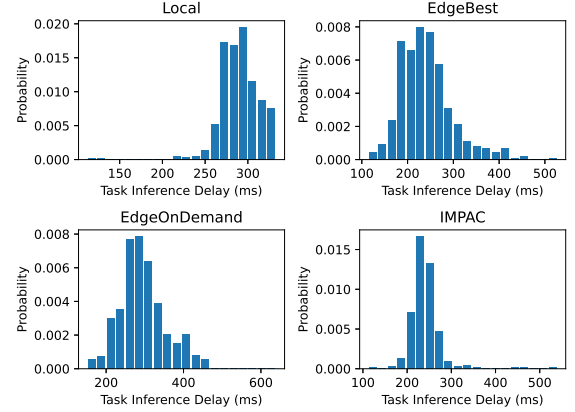


Fig. 5. Frequency of Task Inference Delay for Different IoT Devices

C. Comparison of Task Inference Delay

Fig. 3 shows the average task inference delay along with different time slots for all IoT devices. The average task inference delay incurred by IMPAC, EdgeOnDemand, and EdgeBest would increase along with time slots; due to that, the increasing arrival of IoT devices leads to the reduction of bandwidth and computation resources to each device. We can also find that task average inference delay in the EdgeOnDemand would be larger than that in the EdgeBest. At the initial time stage, we can find that our strategy IMPAC has a similar average task inference delay with the other three strategies. When encountering the excessive number of IoT devices in the MEC platform, IMPAC has a lower average task inference delay over other strategies. Besides, task inference delay is still increasing in the method of EdgeBest and EdgeOnDemand. This is because the number of tasks generated from IoT devices is becoming greater along with time. This may cause resource exhaustion in the edge cloud. So, the delay in the EdgeBest and EdgeOnDemand would increase along with the time slots. Fig. 4 shows the entire distribution of task inference delay for all IoT devices. From Fig. 4, we can find that our strategy can guarantee that task inference delay in most IoT devices is around the latency threshold value of 200ms. Furthermore, we can find that the distribution of task inference delay in our strategy is much denser than the other three strategies. So, our strategy can be more stable to guarantee the

performance of task inference delay following the constraint of task delay. Fig. 5 shows the radio frequency of task inference delay for all IoT devices. In Fig. 5, the task inference delay distribution in the strategy of EdgeBest is more sparse than those in the other three strategies. This is caused by the highly imbalanced resource allocation, in which lots of IoT devices may fall into the resource-hungry situation. To sum up, our strategy can ensure the stability of task inference performance than the other three strategies.

D. Comparison of Rental Cost

Fig. 6 shows the total cumulative rental costs along with different time slots and the number of IoT devices. We can obtain that our strategy can save up to 30% costs than other strategies. For the illustration of EdgeBest, we can acknowledge that excessive resource allocation would lead to a higher rental cost. Instead, by the method of DNN partition, our strategy can save more resource rental costs compared with EdgeOnDemand. From Fig. 6, we can also observe that the growth rate of the total rental cost by IMPAC is lower than that by EdgeOnDemand. IMPAC can utilize the advantage of DNN partition to relieve the pressure of performing all task inference in the edge cloud. So, our strategy can save more rental costs than in EdgeBest.

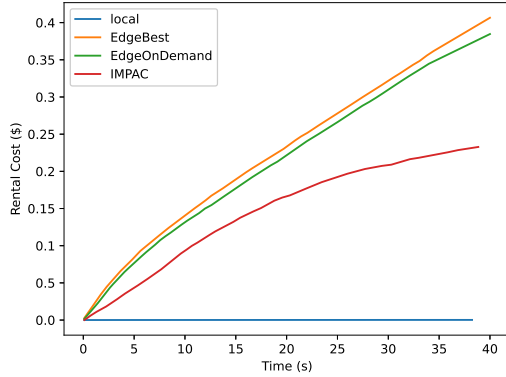


Fig. 6. Total Rental Cost Along with Different Time Slots for Four Strategies

E. Adaptiveness of Our Proposed Strategy

Fig. 7 shows the ratio of DNN partition selection for different IoT devices along with time slots. The four parts with different colors in one bar of the Fig. 7 are classified by the amount of offloading computation. For example, the part named Great in one bar represents the ratio of IoT devices, which select the partitioning point that can offload great computing to the edge cloud. Along with different time slots, the number of IoT devices is constantly growing. Due to the shortage of resources in an edge server and the insufficiency of access networks, IoT devices would gradually prefer to execute their DNN inference locally. In our strategy, we can obtain that each device would decide its own DNN partition point adaptively according to the remaining resource in the

edge cloud. Fig. 8 shows computation resource allocation for IoT devices along with time slots. The unit rental cost of one CPU core is much lower than other resources. Although TPU would generate a higher rental cost, the performance of its computation acceleration is far better than other resources, such as P4 and V100. From Fig. 8, we can observe that two cost-effective resources, CPU and TPU, at different times are highly utilized by IoT devices to ensure the balance between rental cost and task inference delay. To sum up, our strategy can cost-effectively balance the allocation of computing resources from the edge cloud to IoT devices. Based on the above two experimental illustrations, we can obtain that IMPAC can improve the adaptiveness of each IoT device to deal with the optimization of heterogeneous computation resource allocation.

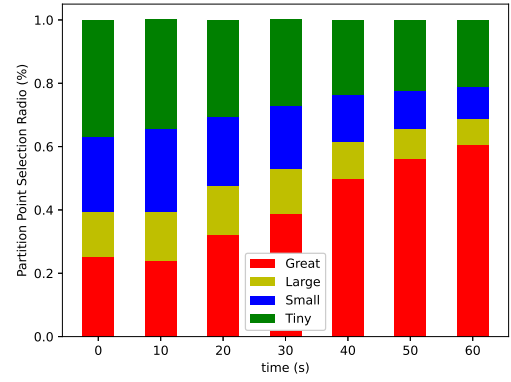


Fig. 7. The Distribution of DNN Partitioning Point Selection Among Different IoT Devices

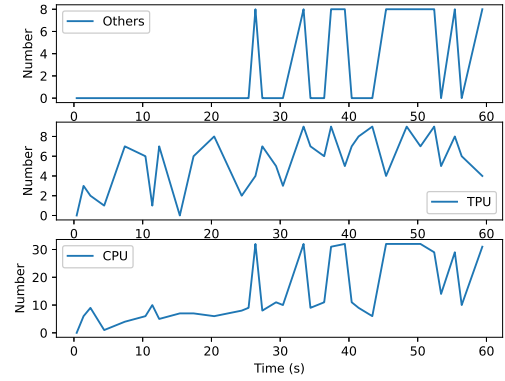


Fig. 8. Heterogeneous Resource Allocation for IoT Devices Along With Different Time Slots

F. Performance Analysis

As the cost-effectiveness and stability of our strategy are mainly determined by the parameter β , which is proved by **Lemma 3** and **Lemma 4**, we illustrate total rental cost and

average task inference delay at different tradeoff parameters β . From Fig. 9, we can obtain that the tradeoff parameter β can change the total rental cost. We can also obtain that the average task inference delay can also be dynamically adjusted by the modification of the tradeoff parameter β . That is to say, the adjustment of tradeoff parameter β can change the total rental cost and the average task inference delay on demand. Moreover, the above results are consistent with the theoretical analysis in the **Lemma 3** and **Lemma 4**. To sum up, our strategy can flexibly meet different service requirements by tuning tradeoff parameter β .

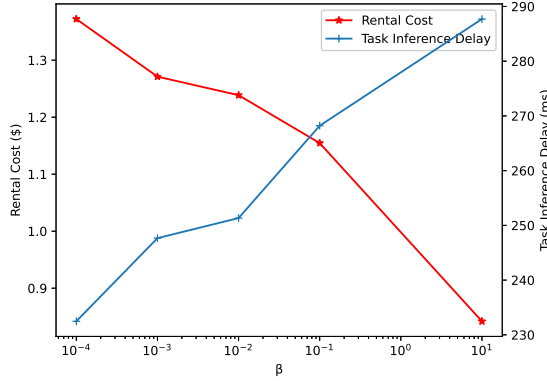


Fig. 9. Impact of Parameter β On task inference delay and rental cost

To illustrate the efficiency of our algorithm, the iterative number in each step of our method is utilized to reflect the convergence of our method. In our performance evaluation for convergence, we set that there are 50 IoT devices that are in the space of our genetic-based method. From Fig. 10, we can obtain that our method would converge in a finite iterative number.

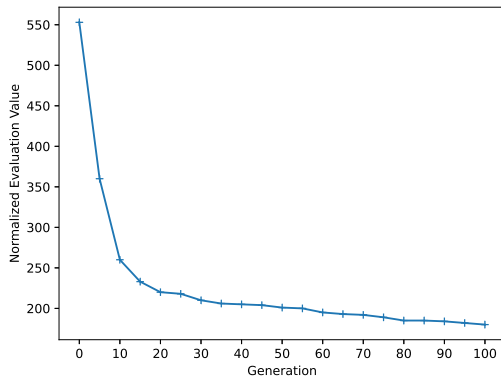


Fig. 10. Convergence of IMPAC

VI. CONCLUSION

In this paper, we demonstrated that DNN-based applications could not be optimized fully by local-only and edge-only

task execution, in terms of rental cost and task processing delay. To improve the quality of DNN-based application in mobile devices, we designed an online algorithm based on lyapunov optimization framework to help mobile devices optimally offload tasks by computation resource allocation and DNN partition. Our proposed strategy can dynamically decide resource allocation and DNN partition for each mobile user, without any future information about wireless network status and arrival of mobile devices. Through a series theoretical analysis, we could show that the rental cost incurred by our strategy can achieve sub-optimally approaching the minimal operational costs within an upper bound, while ensuring stability of DNN inference quality. To validate the theoretical analysis, we developed one discrete-event simulator to conduct trace-driven experiments. Results showed that our strategy can balance the total rental cost and the quality of task inference in the MEC platform.

ACKNOWLEDGMENT

This research has been partly supported by Project U1711264 supported by National Natural Science Foundation of China and by grant 2017YFB1001603 and 2017YFB0202502 in the National Key R&D of China. The authors would like to thank the anonymous reviewers for their helpful and constructive comments and suggestions regarding this manuscript.

APPENDIX A PROOF OF LEMMA 1

Lemma 1. *If the virtual queue Q_k is stable, the average DNN co-inference latency for each IoT device k can satisfy the constraint (14), which can be described as below:*

$$\lim_{t \rightarrow \infty} \frac{Q_k(t)}{t} = 0 \implies \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} T_{k,e}^\theta \leq \epsilon \quad (25)$$

Proof. According to the definition of virtual queue backlog in (16), we can get the inequality below:

$$Q_k(t+1) \geq Q_k(t) + T_{k,e}^\theta - \epsilon$$

By summing the above inequality over the time slot $t \in \{0, 2, \dots, T-1\}$, we can obtain:

$$Q_k(T) - Q_k(0) \geq \sum_{t=0}^{T-1} \{T_{k,e}^\theta - T \cdot \epsilon\}$$

Note that $Q_k(0) = 0$ and $Q_k(T) \geq 0$, we can get that $Q_k(T) - Q_k(0) \geq 0$. Then we can get that the right side of above inequality should follow the constraint as below:

$$\sum_{t=0}^{T-1} \{T_{k,e}^\theta - T \cdot \epsilon\} \leq 0$$

After dividing both sides by T to the above inequality, we can obtain:

$$\frac{1}{T} \sum_{t=0}^{T-1} \{T_{k,e}^\theta - T \cdot \epsilon\} \leq 0$$

By taking $T \rightarrow \infty$, then we can get:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} T_{k,e}^\theta \leq \epsilon$$

□

APPENDIX B PROOF OF LEMMA 2

Lemma 2. *Under any feasible set including partitioning point selection $s_k(e, t)$ and resource allocation strategy $\theta_k = \{d_k^c(n, t), d_k^g(m, t), d_k^u(t)\}$ for all IoT devices, the drift-plus-penalty function **P2** is upper bounded by an expression, which is described as follows:*

$$\begin{aligned} \Delta(Q(t)) + \beta E(Cost_{total}^t | Q(t)) \\ \leq \Gamma + \sum_{k \in K} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon | Q(t)\} \\ + \beta E(Cost_{total}^t | Q(t)), \end{aligned}$$

where $\Gamma = \frac{1}{2} \sum_{k \in K} (T_k^{local} - \epsilon)^2$ is a constant. T_k^{local} is set to be the task inference time incurred by the local-only strategy for IoT device k .

Proof. According to the definition of virtual delay queue, which is described in the (16):

$$Q_k(t+1) = \max\{Q_k(t) + T_{k,e}^\theta - \epsilon, 0\}$$

Then, we can obtain the below expression:

$$Q_k(t+1)^2 \leq \{Q_k(t) + T_{k,e}^\theta - \epsilon\}^2$$

Therefore,

$$\begin{aligned} \frac{Q_k(t+1)^2 - Q_k(t)^2}{2} \\ \leq \frac{1}{2} (Q_k(t) + T_{k,e}^\theta - \epsilon)^2 - Q_k(t)^2 \\ = Q_k(t) \times \{T_{k,e}^\theta - \epsilon\} + \frac{(T_{k,e}^\theta - \epsilon)^2}{2} \end{aligned}$$

According to the constraint (15), DNN inference time incurred by the local-only strategy should be larger than that by other strategies in the process of task co-inference. So, we can get the inequality as below:

$$T_{k,e}^\theta - \epsilon \leq T_k^{local} - \epsilon$$

Based on the above inequality, we can get the below expression:

$$\frac{Q_k(t+1)^2 - Q_k(t)^2}{2} \leq Q_k(t) \times \{T_{k,e}^\theta - \epsilon\} + \frac{1}{2} (T_k^{local} - \epsilon)^2$$

By summing the above inequality over different IoT devices $k \in K$, we can obtain:

$$\begin{aligned} \sum_{k \in K} \frac{Q_k(t+1)^2 - Q_k(t)^2}{2} \\ \leq \sum_{k \in K} Q_k(t) \times \{T_{k,e}^\theta - \epsilon\} + \frac{1}{2} \sum_{k \in K} (T_k^{local} - \epsilon)^2 \end{aligned}$$

According to the definition of Lyapunov drift function:

$$\begin{aligned} L(t) &= \sum_{k \in K} \frac{1}{2} Q_k(t)^2 \\ \Delta(Q(t)) &= L(t+1) - L(t) \end{aligned}$$

Then we can get a bound on $\Delta(Q(t))$:

$$\Delta(Q(t)) \leq \sum_{k \in K} Q_k(t) \times \{T_{k,e}^\theta - \epsilon\} + \Gamma,$$

where $\Gamma = \frac{1}{2} \sum_{k \in K} (T_k^{local} - \epsilon)^2$.

Finally, we add the object function $\beta E(Cost_{total}^t | Q(t))$ to the above inequality. We have:

$$\begin{aligned} \Delta(Q(t)) + \beta E(Cost_{total}^t | Q(t)) \\ \leq \Gamma + \sum_{k \in K} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon | Q(t)\} \\ + \beta E(Cost_{total}^t | Q(t)) \end{aligned}$$

□

APPENDIX C PROOF OF LEMMA 3

Lemma 3. *The average expected total rental cost incurred by IMPAC can achieve to the effect that has a gap with the minimal rental cost, which can be described as below:*

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{-1T} E(Cost_{total}^t) \leq Cost^* + \frac{\Gamma}{\beta},$$

where $Cost^*$ is the minimum of the average rental costs achieved by any joint strategy from the combination of DNN partition and resource allocation under the required constraints.

Proof: From the **Lemma 2**, we can get the inequality below:

$$\begin{aligned} \Delta(Q(t)) + \beta E(Cost_{total}^t | Q(t)) \\ \leq \Gamma + \sum_{k \in K} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon | Q(t)\} \\ + \beta E(Cost_{total}^t | Q(t)) \end{aligned}$$

According to the definition of $\Delta(Q(t))$, we can get the inequality below:

$$\begin{aligned} L(t+1) - L(t) + \beta E(Cost_{total}^t | Q(t)) \\ \leq \Gamma + \sum_{k \in K} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon | Q(t)\} + \beta Cost^*, \end{aligned}$$

In the above expression, $Cost^*$ is obviously larger than $E(Cost_{total}^t|Q(t))$. So, the above inequality can be satisfied in each time slot.

After accumulating the sum of the above inequality over $t \in \{0, 1, 2, \dots, T-1\}$, we can get:

$$\begin{aligned} L(T) - L(0) + \sum_{t=0}^{T-1} \beta E(Cost(c, t)|Q(t)) \\ \leq T \times \Gamma + \sum_{t=0}^{T-1} \sum_{k \in K} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon|Q(t)\} \\ + T \cdot \beta Cost^* \end{aligned}$$

Due to the constraint (14), we can get that $\sum_{k \in K} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon|Q(t)\} \leq 0$. By the pre-defined conditions about $L(0) = 0$ and $L(T) \geq 0$, we can simplify the above inequality and obtain the following as below:

$$\sum_{t=0}^{T-1} \beta E(Cost_{total}^t|Q(t)) \leq T \times \Gamma + T \beta Cost^*$$

After dividing T and β to both sides of above inequality, we can finally get:

$$\frac{1}{T} \sum_{t=0}^{T-1} E(Cost_{total}^t|M_c(t)) \leq \frac{\Gamma}{\beta} + Cost^*$$

APPENDIX D PROOF OF LEMMA 4

Lemma 4. The backlog of the virtual delay queue is stable, which is presented as follows:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{k \in K} E(Q_k(t)) \leq \frac{\Gamma + \beta \cdot (\hat{Cost}_{total}^t - \check{Cost}_{total}^t)}{\delta},$$

where $\delta > 0$, $\hat{Cost}_{total}^t$ and $\check{Cost}_{total}^t$ are defined as the maximal and minimal rental costs in all strategies, respectively.

Proof: We assume that there exists one strategy that satisfies the average delay constraint (14), and we assume there exists one parameter $\alpha > 0$ that follows:

$$\epsilon \geq T_{k,e}^\theta + \alpha, \forall k \in K$$

According to the previous definition, we can utilize $Cost_{total}^t$ as the total rental cost for this strategy. And, we can also acknowledge that $\check{Cost}_{total}^t \leq Cost^t \leq \hat{Cost}_{total}^t$.

According to the **Lemma 3**, we can obtain the below expression in the proof of **Lemma 3**,

$$\begin{aligned} L(t+1) - L(t) + \beta E(Cost_{total}^t|Q(t)) \\ \leq \Gamma + \sum_{t=0}^{T-1} \sum_{k \in K} Q_k(t) \cdot E\{T_{k,e}^\theta - \epsilon|Q(t)\} \\ + \beta Cost^*, \end{aligned}$$

Then, summing over $t \in \{0, 1, 2, 3, \dots, T-1\}$ and get:

$$\begin{aligned} L(T) - L(0) + \sum_{t=0}^{T-1} \beta E(Cost_{total}^t|Q(t)) \\ \leq T \times \Gamma - \sum_{t=0}^{T-1} \sum_{k \in K} Q_k(t) \times \alpha + T \cdot \beta Cost^* \end{aligned}$$

After dividing T and α to both sides of above inequality, we can finally get:

$$\begin{aligned} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{k \in K} Q_k(t) \leq \frac{\Gamma + \beta(Cost_{total}^t - \frac{1}{T} \sum_{t=0}^{T-1} E(Cost_{total}^t|Q(t)))}{\alpha} \\ + \frac{E(L(0))}{\alpha T} - \frac{E(L(T))}{\alpha T} \end{aligned}$$

From the definition of $\hat{Cost}_{total}^t$ and $\check{Cost}_{total}^t$ and the predefined value $L(0)$, we can get one expression as below:

$$Cost_{total}^t - \frac{1}{T} \sum_{t=0}^{T-1} E(Cost_{total}^t) \leq \hat{Cost}_{total}^t - \check{Cost}_{total}^t$$

To sum up, we can get the conclusion by the replacement from the last two expressions to third from last one:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{k \in K} \sum_{t=0}^{T-1} E(Q_k(t)) \leq \frac{\Gamma + \beta \cdot (\hat{Cost}_{total}^t - \check{Cost}_{total}^t)}{\delta},$$

REFERENCES

- [1] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.
- [2] S. Kuutti, R. Bowden, Y. Jin, P. Barber, and S. Fallah, "A survey of deep learning applications to autonomous vehicle control," *IEEE Transactions on Intelligent Transportation Systems*, 2020.
- [3] L. Rui, Y. Zhu, Z. Gao, and X. Qiu, "CLPM: A Cooperative Link Prediction Model for Industrial Internet of Things Using Partitioned Stacked Denoising Autoencoder," *IEEE Transactions on Industrial Informatics*, 2020.
- [4] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi *et al.*, "A configurable cloud-scale DNN processor for real-time AI," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 1–14.
- [5] Y. Chen, N. Zhang, Y. Zhang, X. Chen, W. Wu, and X. S. Shen, "TOFFEE: Task offloading and frequency scaling for energy efficiency of mobile devices in mobile edge computing," *IEEE Transactions on Cloud Computing*, 2019.
- [6] Y. Wang, X. Tao, X. Zhang, P. Zhang, and Y. T. Hou, "Cooperative task offloading in three-tier mobile computing networks: An ADMM framework," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 3, pp. 2763–2776, 2019.
- [7] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration," *IEEE Communications Surveys Tutorials*, vol. 19, no. 3, pp. 1657–1681, 2017.
- [8] D. Xu, T. Li, Y. Li, X. Su, S. Tarkoma, and P. Hui, "A Survey on Edge Intelligence," *arXiv preprint arXiv:2003.12172*, 2020.
- [9] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

- [10] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 450–465, 2017.
- [11] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795–2808, 2015.
- [12] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive DNN surgery for inference acceleration on the edge," in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IEEE, 2019, pp. 1423–1431.
- [13] S. Sheng, L. Xujing, L. Min, Y. Bo, and G. Xiaobing, "DNN Inference Acceleration via Heterogeneous IoT Devices Collaboration," *Journal of Computer Research and Development*, vol. 57, no. 4, p. 709, 2020.
- [14] J. H. Ko, T. Na, M. F. Amir, and S. Mukhopadhyay, "Edge-host partitioning of deep neural networks with feature space encoding for resource-constrained internet-of-things platforms," in *2018 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*. IEEE, 2018, pp. 1–6.
- [15] B. Lin, Y. Huang, J. Zhang, J. Hu, X. Chen, and J. Li, "Cost-Driven Off-Loading for DNN-Based Applications Over Cloud, Edge, and End Devices," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 8, pp. 5456–5466, 2019.
- [16] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. Von Kistowski, A. Ali-eldin, C. Abad, J. N. Amaral, P. Tuma, and A. Iosup, "Methodological principles for reproducible performance evaluation in cloud computing," *IEEE Transactions on Software Engineering*, pp. 1–1, 2019.
- [17] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "JointDNN: an efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Transactions on Mobile Computing*, 2019.
- [18] M. Xu, F. Qian, M. Zhu, F. Huang, S. Pushp, and X. Liu, "Deepwear: Adaptive local offloading for on-wearable deep learning," *IEEE Transactions on Mobile Computing*, vol. 19, no. 2, pp. 314–330, 2019.
- [19] J. Fowers, G. Brown, P. Cooke, and G. Stitt, "A performance and energy comparison of FPGAs, GPUs, and multicores for sliding-window applications," in *Proceedings of the ACM/SIGDA international symposium on Field Programmable Gate Arrays*, 2012, pp. 47–56.
- [20] J. Baek and G. Kaddoum, "Heterogeneous Task Offloading and Resource Allocations via Deep Recurrent Reinforcement Learning in Partial Observable Multi-Fog Networks," *IEEE Internet of Things Journal*, 2020.
- [21] Z. Cao, P. Zhou, R. Li, S. Huang, and D. Wu, "Multi-Agent Deep Reinforcement Learning for Joint Multi-Channel Access and Task Offloading of Mobile Edge Computing in Industry 4.0," *IEEE Internet of Things Journal*, 2020.
- [22] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 615–629, 2017.
- [23] W. Ju, D. Yuan, W. Bao, L. Ge, and B. B. Zhou, "DeepSave: saving DNN inference during handovers on the edge," in *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*, 2019, pp. 166–178.
- [24] M. J. Neely, "Stochastic network optimization with application to communication and queueing systems," *Synthesis Lectures on Communication Networks*, vol. 3, no. 1, pp. 1–211, 2010.
- [25] C.-C. Hsu, C.-K. Yang, J.-J. Kuo, W.-T. Chen, and J.-P. Sheu, "Cooperative convolutional neural network deployment over mobile networks," in *ICC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 2020, pp. 1–7.
- [26] M. Gao, W. Cui, D. Gao, R. Shen, J. Li, and Y. Zhou, "Deep Neural Network Task Partitioning and Offloading for Mobile Edge Computing," in *2019 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2019, pp. 1–6.
- [27] T. Mohammed, C. Joe-Wong, R. Babbar, and M. Di Francesco, "Distributed Inference Acceleration with Adaptive DNN Partitioning and Offloading," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 854–863.
- [28] S. Q. Zhang, J. Lin, and Q. Zhang, "Adaptive Distributed Convolutional Neural Network Inference at the Network Edge with ADCNN," in *49th International Conference on Parallel Processing-ICPP*, 2020, pp. 1–11.
- [29] L. Zhou, H. Wen, R. Teodorescu, and D. H. Du, "Distributing deep neural networks with containerized partitions at the edge," in *2nd {USENIX} Workshop on Hot Topics in Edge Computing (HotEdge 19)*, 2019.
- [30] X. Ran, H. Chen, X. Zhu, Z. Liu, and J. Chen, "Deepdecision: A mobile deep learning framework for edge video analytics," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 1421–1429.
- [31] Z. Ning, P. Dong, X. Wang, X. Hu, J. Liu, L. Guo, B. Hu, R. Kwok, and V. C. M. Leung, "Partial Computation Offloading and Adaptive Task Scheduling for 5G-enabled Vehicular Networks," *IEEE Transactions on Mobile Computing*, pp. 1–1, 2020.
- [32] E. Li, Z. Zhou, and X. Chen, "Edge intelligence: On-demand deep learning model co-inference with device-edge synergy," in *Proceedings of the 2018 Workshop on Mobile Edge Communications*, 2018, pp. 31–36.
- [33] A. E. Eshratifar and M. Pedram, "Energy and performance efficient computation offloading for deep neural networks in a mobile cloud computing environment," in *Proceedings of the 2018 on Great Lakes Symposium on VLSI*, 2018, pp. 111–116.
- [34] X. Tang, X. Chen, L. Zeng, S. Yu, and L. Chen, "Joint Multi-User DNN Partitioning and Computational Resource Allocation for Collaborative Edge Intelligence," *IEEE Internet of Things Journal*, 2020.
- [35] Z. Wang, X. Tang, Q. Liu, and J. Han, "Jily: Cost-Aware AutoScaling of Heterogeneous GPU for DNN Inference in Public Cloud," in *2019 IEEE 38th International Performance Computing and Communications Conference (IPCCC)*, 2019, pp. 1–8.
- [36] G. Yeung, D. Borowiec, A. Friday, R. Harper, and P. Garraghan, "Towards GPU Utilization Prediction for Cloud Deep Learning," in *12th {USENIX} Workshop on Hot Topics in Cloud Computing (HotCloud 20)*, 2020.
- [37] A. Fischer, J. F. Botero, M. T. Beck, H. de Meer, and X. Hesselbach, "Virtual Network Embedding: A Survey," *IEEE Communications Surveys Tutorials*, vol. 15, no. 4, pp. 1888–1906, 2013.
- [38] H. Yao, S. Ma, J. Wang, P. Zhang, C. Jiang, and S. Guo, "A Continuous-Decision Virtual Network Embedding Scheme Relying on Reinforcement Learning," *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 864–875, 2020.
- [39] Z. Yan, J. Ge, Y. Wu, L. Li, and T. Li, "Automatic Virtual Network Embedding: A Deep Reinforcement Learning Approach With Graph Convolutional Networks," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 6, pp. 1040–1057, 2020.
- [40] P. A. Apostolopoulos, E. E. Tsiropoulou, and S. Papavassiliou, "Risk-Aware Data Offloading in Multi-Server Multi-Access Edge Computing Environment," *IEEE/ACM Transactions on Networking*, vol. 28, no. 3, pp. 1405–1418, 2020.
- [41] Q. Lu, X. Zheng, J. Ma, Y. Dong, Z. Qi, J. Yao, B. He, and H. Guan, "gMig: Efficient vGPU Live Migration with Overlapped Software-Based Dirty Page Verification," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 5, pp. 1209–1222, 2019.
- [42] S. Hu, C. Dong, and W. Wen, "Enable Pipeline Processing of DNN Co-inference Tasks in the Mobile-Edge Cloud," in *2021 IEEE 6th International Conference on Computer and Communication Systems (ICCCS)*, 2021, pp. 186–192.
- [43] S. Zhou and W. Jadoon, "The partial computation offloading strategy based on game theory for multi-user in mobile edge computing environment," *Computer Networks*, p. 107334, 2020.
- [44] Y. Zhang, J.-H. Liu, C.-Y. Wang, and H.-Y. Wei, "Decomposable Intelligence on Cloud-Edge IoT Framework for Live Video Analytics," *IEEE Internet of Things Journal*, 2020.
- [45] X. Xu, Q. Liu, Y. Luo, K. Peng, X. Zhang, S. Meng, and L. Qi, "A computation offloading method over big data for IoT-enabled cloud-edge computing," *Future Generation Computer Systems*, vol. 95, pp. 522–533, 2019.
- [46] I. Lera, C. Guerrero, and C. Juiz, "YAFS: A Simulator for IoT Scenarios in Fog Computing," *IEEE Access*, vol. 7, pp. 91 745–91 758, 2019.
- [47] S. Jošilo and G. Dán, "Wireless and computing resource allocation for selfish computation offloading in edge computing," in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IEEE, 2019, pp. 2467–2475.



Chongwu Dong (dongchw3@mail.sysu.edu.cn) received his B.S. degree in software engineering from School of Software, Sun Yat-sen University, Guangzhou, China in 2013, and Ph.D. degree in computer science from School of Data and Computer Science, Sun Yat-sen University, Guangzhou, China in 2013 in 2019. Now, he is a Post-Doctoral Research Fellow at the School of Computer Science And Engineering, Sun Yat-sen University, Guangzhou, China. His research interests include cloud architecture, edge computing, wireless communication and D2D network.



Sheng Hu (hush26@mail2.sysu.edu.cn) received his B.S. degree in School of Data and Computer Science, Sun Yat-sen University, Sun Yat-sen University, Guangzhou, China in 2019. Currently he is a M.S. student at School of Computer Science And Engineering, Sun Yat-sen University. His research interests focus on deep learning, edge computing and reinforcement Learning.



Xi Chen (chenx636@mail2.sysu.edu.cn) received his B.S. degree in School of Data and Computer Science, Sun Yat-sen University, Sun Yat-sen University, Guangzhou, China in 2019. Currently he is a M.S. student at School of Computer Science And Engineering, Sun Yat-sen University. His research interests focus on cloud computing, edge computing and nature language processing.



Wushao Wen is a professor in the School of Data and Computer Science, Sun Yat-Sen University, P.R. China. He received the B.S. degree from the University of Science and Technology of China in 1993 and the M.S. and Ph.D. degrees from the University of California, Davis, in 1999 and 2001, respectively. He was an Engineer and Project Manager with China Telecommunication Inc, from 1993 to 1997. From 2000 to 2001, he was an engineer with Cisco Systems, Inc, USA. From 2001 to 2004, he was a Senior Engineer at CIENA Corporation, Cupertino,

CA, leading the design and implementation of optical routing and signaling systems. He worked as a senior networking expert in McAfee Inc from 2004 to 2006 and a staff engineer in Juniper Networks from 2006 to 2009 in network intrusion detection area. He has been a professor in Sun Yat-sen University since 2009 and was appointed as the Chief Director of the Networks and Information Center, the Director of Shared Experimentation Teaching Center, Sun Yat-Sen University, P.R. China in 2013, 2014 respectively. He is currently doing research in cloud computing, network security, network architectures, and multimedia networks.